

Lekcja 09 - Ecosystem: boto3, Avro i API

2026-08-21

Ekosystem: boto3, Avro i API

```
pobranie -> serializacja -> zapis -> walidacja ->
magazyn
```

Do tej pory pracowaliśmy głównie lokalnie. Teraz Python robi to, do czego jest dobrze przygotowany: łączy zewnętrzne źródła danych z narzędziami magazynowania i kontrolą jakości.

Realny pipeline danych wygląda zwykle tak:

```
API / eksport / event -> Python -> walidacja ->  
serializacja -> magazyn
```

Najważniejsza umiejętność to nie „umieć zrobić request”. To umieć rozpoznać:

```
co jest błędem przejściowym,  
co jest problemem kontraktu,  
co trzeba zapisać jako artefakt,  
co jest gotowe do dalszej analityki.
```

■ Gdzie dane pochodzą w realnym świecie

W data engineering dane zwykle pochodzą z:

```
API partnerów  
S3 / object storage  
streaming events  
eksportów z systemów operacyjnych
```

Python jest warstwą łączącą: zbiera, waliduje, serializuje i przenosi dane do odpowiedniego magazynu.

■ Dlaczego to jest złożone

Sama odpowiedź z API to nie jest pipeline. To tylko wejście.

- API ma limity szybkości
- odpowiedź ma paginację
- dane mogą mieć różne typy i nulls
- serializacja musi być stabilna
- zapis do storage ma własne reguły błędów

Dlatego zajęcia z ekosystemem są momentem przejścia od „czytam JSON” do „obsługuję realny ingest”.

■ Wzorzec integracji

Każdy zewnętrzny system ma podobny przepływ:

```
klent -> wywołanie -> obsługa odpowiedzi -> retry  
przy błędzie przejściowym -> zapis lub upload
```

Najważniejsze: nie retryujesz wszystkiego. Retry jest zarezerwowany tylko dla błędów przejściowych, a nie dla problemów jakości danych.

```
while True:
    response = client.get(url, params=params)
    response.raise_for_status()
    payload = response.json()
    records.extend(payload["results"])

    if not payload.get("next"):
        break
    params["cursor"] = payload["next"]
```

W praktyce API zwykle nie zwraca wszystkiego naraz. Musisz obsłużyć `next`, `cursor` albo `page` i iterować po serii odpowiedzi.

■ Typowe błędy przy API

- 429 limit szybkości
- 500 / 502 / 503 timeout
- 400 niepoprawne zapytanie
- 401/403 problem z autoryzacją
- brak tokena paginacji
- częściowa odpowiedź ze zmienionym schematem

Nie wszystkie błędy są takie same. To wpływa na to, czy robimy retry, czy natychmiast zgłaszamy błąd kontraktu.

```
import boto3

s3 = boto3.client("s3", region_name="eu-west-1")

s3.upload_file("output/orders.parquet", "my-bucket",
               "orders/orders.parquet")
s3.put_object(Bucket="my-bucket", Key="orders/data.avro", Body=avro_bytes)
```

W praktyce klient bucketa tworzy się raz, a potem używa wielokrotnie. To jest prosty, ale realny wzorzec pracy z magazynem w chmurze.

```
from botocore.exceptions import ClientError

try:
    s3.download_file(bucket, key, local_path)
except ClientError as exc:
    code = exc.response["Error"]["Code"]
    if code == "NoSuchKey":
        raise FileNotFoundError(f"Missing: s3://{
{bucket}/{key}") from exc
    raise
```

Nie ma jednego „ogólnego błędu”. Dla S3 ważne są szczegóły odpowiedzi, bo różne kody oznaczają różne decyzje operacyjne.

Avro: row-based, dobre do strumieni i payloady eventów

Parquet: columnar, dobre do analityki i zapytań w data lake

W tej lekcji najczęściej pracujemy z Avro jako formatem serializacji między etapami ingest. Parquet z kolei zwykle pojawia się w projekcie praktycznym.

```
import fastavro
from io import BytesIO

schema = {
    "type": "record",
    "name": "Order",
    "fields": [
        {"name": "order_id", "type": "string"},
        {"name": "status", "type": "string"},
        {"name": "total_amount", "type": "double"}
    ],
}

records = [{"order_id": "1001", "status": "completed", "total_amount": 120.5}]
buf = BytesIO()
fastavro.writer(buf, fastavro.parse_schema(schema), records)
```

Schema robi za nas kontrakt danych na poziomie serializacji. To ważne, bo format pliku ma znaczenie dla downstream.

■ Różnica: pobieranie / serializacja / zapis

```
pobieranie: pobierz dane z API  
serializacja: zamień na Avro / JSON / bytes  
zapis: zapisz do S3 / lokalnie / do kolejnego systemu
```

To jest podstawowy wzorzec pipeline ingest. Jeśli zrobisz tylko pobieranie, nie zrobisz jeszcze gotowego przepływu danych.

■ Kiedy retry ma sens

Retry tylko dla błędów przejściowych:

```
429 (limit szybkości)
408 (timeout)
5xx (przejściowa awaria serwera)
```

Nie retry dla błędów stałych:

```
400 (złe zapytanie)
401/403 (brak autoryzacji)
404 (zły endpoint)
```

Dobłą regułą jest: retry tylko tam, gdzie problem jest czasowy i ma sens go powtórzyć.

■ Backoff i jitter

Nie spamujesz API co 100 ms. Zamiast tego używasz rosnących opóźnień:

```
próba 1 -> 0.5s  
próba 2 -> 1.0s  
próba 3 -> 2.0s  
próba 4 -> 4.0s
```

Jitter rozprasza równoległe retry, więc wiele workerów nie wraca do tego samego endpointu w tej samej chwili.

■ Co jest „wystarczająco dobrym” pipeline

- czyta zewnętrzne źródło
- obsługuje paginację
- waliduje dane po wejściu
- serializuje do stabilnego formatu
- zapisuje do magazynu
- ma logi i politykę retry

To jest minimum, które odróżnia realny ingest od prostego „request + print”.

Definition of Done

- [] payload API jest pobierany i paginowany
- [] dane są serializowane do Avro lub innego formatu
- [] wynik trafia do magazynu
- [] błędy są rozpoznawane po kodach i typach
- [] retry jest tylko dla błędów przejściowych
- [] projekt ma jasno opisany przepływ i kontrakt danych

To jest minimum, które odróżnia realny ingest od prostego „request + print”.

■ Odpowiedź na pytanie rekrutacyjne

Pytanie:

Jak zbudować pipeline ingest w Pythonie dla danych zewnętrznych?

Odpowiedź:

Pobieram dane z API z paginacją, waliduję surowe rekordy i serializuję je do stabilnego formatu, na przykład Avro.
Następnie zapisuję artefakt do magazynu obiektowego przez boto3.
Trzymam retry tylko dla błędów przejściowych i traktuję trwałe problemy ze schematem jako problemy kontraktu.

Checklist do powtórki

- [] Czy potrafisz opisać różnicę między pobieraniem, serializacją i zapisem?
- [] Czy rozumiesz, kiedy retry ma sens, a kiedy nie?
- [] Czy wiesz, po co używamy Avro / Parquet?
- [] Czy potrafisz powiedzieć, czym jest paginacja w API?
- [] Czy rozumiesz, że Python w data engineering jest warstwą łączącą, nie tylko językiem skryptowym?

W DE to kluczowe, bo duplikaty psuja metryki.

■ Mocking w testach

Zewnętrzne systemy (S3, API) nie powinny być wywoływane w testach.

```
from unittest.mock import MagicMock

def test_upload_calls_put_object() -> None:
    mock_s3 = MagicMock()
    upload_to_s3(b"data", bucket="my-bucket", key
    ="k", client=mock_s3)
    mock_s3.put_object.assert_called_once_with(
        Bucket="my-bucket", Key="k", Body=b"data"
    )
```

Klient jako parametr = testowalność bez prawdziwego AWS.

Klient w środku funkcji = nie da się przetestować bez moto/localstack.

Pytanie:

How do you handle external integrations in a Python DE pipeline?

Odpowiedz:

I treat each external system as a boundary.
The client is created once per session, not per operation.
IO functions accept the client as a parameter – this makes them testable.
In tests I replace real clients with MagicMock and verify the call arguments.
For S3 I use boto3 with ClientError handling.
For REST APIs I use httpx with cursor-based pagination and bounded retries.
Retry is only for transient errors with exponential backoff and jitter.

Oddajesz:

```
src/ingestion/  
  api.py      ← fetch z paginacja +  
retry/backoff  
  serialize.py ← Avro roundtrip  
  storage.py  ← upload z klientem jako parametr  
tests/  
  test_api.py  
  test_serialize.py  
  test_storage.py
```

Najwazniejsze:

```
testy nie robia prawdziwych HTTP ani S3 calls  
retry dziala tylko dla transient statusow  
mypy src/ – czysto
```